

Finly Azure Hosting Guide (Detailed)

1. Objective

This document records the complete Azure hosting workflow used to deploy Finly in production, including architecture, commands, fixes applied during rollout, verification, and operations guidance.

This guide is intended for maintainers who need to reproduce or operate the deployment.

2. Deployed Architecture

- **Application runtime:** FastAPI + static frontend assets from the same container
- **Compute:** Azure Container Apps (Consumption)
- **Database:** Azure SQL Database
- **Container registry:** Docker Hub (nikhil251200/finly)
- **Routing:** Public ingress on Container App (HTTPS)

3. Azure Resources Used

- **Subscription:** Azure subscription 1
- **Resource group:** finly-rg
- **Region:** centralindia
- **Container Apps environment:** finly-env
- **Container App:** finly-app
- **Public URL:** <https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io>
- **Azure SQL Server:** finlysql29987
- **Azure SQL Database:** finlydb

4. Prerequisites

- Azure CLI authenticated (`az login`) and correct subscription selected
- Docker with Buildx support
- Docker Hub login (`docker login`)
- Source includes `Dockerfile` and `.dockerignore`

5. Provider Registration (SQL)

SQL creation fails if `Microsoft.Sql` is not registered.

```
az account show
az provider register --namespace Microsoft.Sql
az provider show --namespace Microsoft.Sql --query registrationState -o tsv
```

Expected status: Registered

6. Build and Push Container Image

Container Apps required an amd64-compatible image in this deployment.

```
docker buildx build --platform linux/amd64 -t nikhil251200/finly:latest --push .
```

7. SQL Setup

```
RG="finly-rg"
SQL_SERVER="finlysql29987"
DB_NAME="finlydb"
SQL_ADMIN="finlyadmin"
```

```
# Generate local strong password (do not commit)
```

```
SQL_PASSWORD="$(python3 -c 'import secrets,string; chars=string.ascii_letters+string.digits+string.punctuation; print('".join(random.choice(chars) for _ in range(32))')")"
```

```
az sql server update -g "$RG" -n "$SQL_SERVER" -p "$SQL_PASSWORD"
```

```
az sql db create -g "$RG" -s "$SQL_SERVER" -n "$DB_NAME" --edition Basic
```

```
# Allow Azure services and current client IP
```

```
az sql server firewall-rule create -g "$RG" -s "$SQL_SERVER" -n AllowAzure --start-ip-address 0.0.0.0 --end-ip-address 255.255.255.255
MYIP="$(curl -s https://ifconfig.me)"
```

```
az sql server firewall-rule create -g "$RG" -s "$SQL_SERVER" -n AllowMyCurrentIp --start-ip-address $MYIP --end-ip-address $MYIP
```

8. Container App Create/Configure

```
RG="finly-rg"
APP_NAME="finly-app"
ENV_NAME="finly-env"
IMAGE="docker.io/nikhil251200/finly:latest"
SQL_SERVER="finlysql29987"
SQL_ADMIN="finlyadmin"
DB_NAME="finlydb"
JWT_SECRET="$(python3 -c 'import secrets; print(secrets.token_urlsafe(48))')"
```

```
az containerapp create \
  -g "$RG" \
  -n "$APP_NAME" \
  --environment "$ENV_NAME" \
  --image "$IMAGE" \
  --ingress external \
  --target-port 8000 \
  --min-replicas 0 \
  --max-replicas 1 \
  --cpu 0.5 \
  --memory 1.0Gi \
  --secrets db-password="$SQL_PASSWORD" jwt-secret="$JWT_SECRET" \
```

```
--env-vars \
  APP_ENV=production \
  DB_SERVER="$SQL_SERVER.database.windows.net" \
  DB_PORT=1433 \
  DB_NAME="$DB_NAME" \
  DB_USER="$SQL_ADMIN" \
  DB_PASSWORD=secretref:db-password \
  DB_ENCRYPT=yes \
  DB_TRUST_SERVER_CERTIFICATE=no \
  JWT_SECRET=secretref:jwt-secret \
  CORS_ALLOWED_ORIGINS="https://finly-app.lemonflower-e7039555.centralindia.azurecontainer"
```

9. Database Schema Initialization

In this rollout, schema initialization was done with `docker run` against Azure SQL.

```
docker run --rm --platform linux/amd64 \
  -e APP_ENV=production \
  -e DB_SERVER="finlysql29987.database.windows.net" \
  -e DB_PORT=1433 \
  -e DB_NAME="finlydb" \
  -e DB_USER="finlyadmin" \
  -e DB_PASSWORD="$SQL_PASSWORD" \
  -e DB_ENCRYPT=yes \
  -e DB_TRUST_SERVER_CERTIFICATE=no \
  -e JWT_SECRET="temp-init-secret" \
  -e CORS_ALLOWED_ORIGINS="*" \
  nikhil251200/finly:latest \
  python -c "from app.core.db_init import init_database_schema; init_database_schema(); print"
```

10. SMTP (Gmail) Configuration

```
az containerapp secret set -g finly-rg -n finly-app --secrets smtp-password="<gmail-app-pass"
```

```
az containerapp update -g finly-rg -n finly-app --set-env-vars \
  SMTP_HOST="smtp.gmail.com" \
  SMTP_PORT="587" \
  SMTP_USER="<your-email>" \
  FROM_EMAIL="<your-email>" \
  SMTP_PASSWORD=secretref:smtp-password \
  DEBUG_RESET_CODES="0" \
  DEPLOYED_AT="$(date +%s)"
```

11. Deterministic Deployment (Recommended)

Use image digest for predictable releases:

```
az containerapp update -g finly-rg -n finly-app \
  --image "docker.io/nikhil251200/finly@sha256:<digest>" \
  --set-env-vars DEPLOYED_AT="$(date +%s)"
```

12. Verification Checklist

12.1 Platform status

```
az containerapp show -g finly-rg -n finly-app --query "{fqdn:properties.configuration.ingress.fqdn}"
az containerapp revision list -g finly-rg -n finly-app -o table
```

12.2 Endpoint checks

```
curl -sS https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io/health
curl -sS -o /dev/null -w "%{http_code}" https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io/health
curl -sS -o /dev/null -w "%{http_code}" https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io/health
curl -sS -o /dev/null -w "%{http_code}" https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io/health
curl -sS -o /dev/null -w "%{http_code}" https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io/health
```

12.3 Logs

```
az containerapp logs show -g finly-rg -n finly-app --tail 200
```

13. Issues Encountered and Resolutions

13.1 SQL provider registration error

- **Symptom:** MissingSubscriptionRegistration for Microsoft.Sql
- **Fix:** register provider using Azure CLI.

13.2 Docker build failure (apt-key)

- **Symptom:** build failed using deprecated key flow.
- **Fix:** switched Dockerfile to Debian Bookworm keyring + signed repository configuration.

13.3 Image architecture mismatch

- **Symptom:** Container App revision could not start from ARM image.
- **Fix:** rebuilt and pushed with --platform linux/amd64.

13.4 Root/app/admin route runtime 500s

- **Symptom:** /, /app, /admin returned server errors.
- **Cause:** expected frontend files were absent in image.
- **Fix:** ensured Frontend/index.html, Frontend/app.html, and Frontend/admin.html were included and redeployed.

13.5 Password reset email not received

- **Symptom:** forgot-password API returned success but no inbox email.
- **Cause:** SMTP not configured in production.
- **Fix:** added Gmail SMTP env + secret and verified successful send in logs.

14. Operational Runbook

14.1 Rotate SQL password safely

1. Update Azure SQL server password.
2. Update Container App secret `db-password`.
3. Trigger new revision by env update (`DEPLOYED_AT`).
4. Run health/auth smoke checks.

14.2 Rotate JWT secret safely

1. Set new `jwt-secret` in Container App secrets.
2. Roll revision.
3. Expect existing sessions to re-authenticate.

14.3 Rollback strategy

1. Identify previous healthy revision/image digest.
2. Redeploy prior digest via `az containerapp update --image ....`
3. Validate health and key routes.

15. Security Notes

- Do not store plaintext secrets in repo or documentation.
- Store all sensitive values in Azure Container App secrets.
- Keep `DEBUG_RESET_CODES=0` in production except controlled diagnostics.
- Restrict `CORS_ALLOWED_ORIGINS` to trusted domains only.

16. References

- `docs/azure-deploy.md`
- `docs/ENGINEERING_DOCUMENTATION.md`
- `docs/API_CONTRACT.md`
- `Dockerfile`
- `app/main.py`